

Structures de contrôle

Corrigé

Notions acquises à l'issue du TP :

- Savoir choisir la « bonne » structure de contrôle dans un algorithme.

Exercice 1 : Structures de contrôle	1
Exercice 2 : Racine carrée d'un nombre (Méthode de Newton)	1
Exercice 3 : Puissance	2
Exercice 4 : Amélioration du calcul de la puissance entière	4

Exercice 1 : Structures de contrôle

L'objectif de cet exercice est de manipuler les structures de contrôle de notre pseudo-langage algorithmique et leur pendant en Ada. Les programmes listés ci-après sont à compléter dans l'ordre. Chaque programme commence par une description de son objectif et des exemples d'utilisation qui vous permettront de tester vos programmes. Il faudra faire attention à bien utiliser la bonne structure de contrôle.

On ne traitera pas la robustesse de ces programmes : on considère que l'utilisateur saisira toujours une donnée valide.

1. permuter_caracteres.adb
2. tarif_place.adb
3. classer_caractere.adb
4. compte_jules_objectif.adb
5. chiffre_significatif.adb
6. table_7.adb
7. table_pythagore.adb
8. score_21.adb
9. somme_serie_double.adb

Pour aller plus loin, pour les plus rapides ou ceux qui ont envie de s'entraîner voici quelques exercices supplémentaires et optionnels.

Exercice 2 : Racine carrée d'un nombre (Méthode de Newton)

La k^{ieme} approximation de la racine carrée de x est donnée par $a_{k+1} = (a_k + x/a_k)/2$ et $a_0 = 1$.

On arrête le calcul quand la distance entre a_{k+1} et a_k est inférieure à une précision donnée.

1. Écrire un programme (fichier `newton.adb`) qui affiche une valeur approchée de la racine carrée d'un nombre en utilisant la méthode précédente. Nombre et précision seront lus au clavier.
2. On peut aussi arrêter le calcul des a_k quand a_k^2 est proche de x à la précision près. Ajouter cette nouvelle approche dans le programme précédent.

Exercice 3 : Puissance

Afficher la puissance entière d'un réel en utilisant somme et multiplication (`puissance.adb`). On traite d'abord le cas où l'exposant est positif avant de généraliser aux entiers relatifs.

Solution :

Dans la solution de cet exercice, nous utilisons la méthode des raffinages qui sera l'objet du prochain cours. Ceci permet de donner un exemple de son application. Pour la réponse à cet exercice, c'est le programme qui en découle qui est attendu.

On peut, dans un premier temps, se demander si la puissance entière d'un réel a toujours un sens. En fait, ceci n'a pas de sens si le nombre est nul et si l'exposant est strictement négatif.

```
1      2 puissance -3    -> 0.125 -- cas nominal (puissance négative)
```

On choisit de contrôler la saisie de manière à garantir que nous ne sommes pas dans l'un de ces cas. Nous souhaitons donner à l'utilisateur un message le plus clair possible lorsque la saisie est invalide.

Nous envisageons les jeux de tests suivants :

```
1  nombre exposant  ->     $x^n$ 
2
3      2          3    ->    8 -- cas nominal
4      3          2    ->    9 -- cas nominal
5      1          1    ->    1 -- cas nominal
6      2         -3    -> 0.125 -- cas nominal (exposant négative)
7      0          2    ->    0 -- cas nominal (x est nul)
8      0          0    ->    1 -- par convention
9      1.5        2    -> 2.25 -- x peut être réel
10     0         -2    -> ERREUR -- division par zéro
```

Voyons maintenant le principe de la solution. Par exemple, pour calculer 2^3 , on peut faire $2 * 2 * 2$. Plus généralement, on multiplie n fois x par lui-même (donc une boucle **Pour**).

Si on essaie ce principe, on constate qu'il ne fonctionne pas dans le cas d'une puissance négative. Prenons le cas de 2^{-3} . On peut le réécrire $(1/2)^3$. On est donc ramené au cas précédent. Le facteur multiplicatif est dans ce cas $1/x$ et le nombre de fois est $-n$.

Nous introduisons donc deux variables intermédiaires qui sont :

```
facteur: Réel    -- facteur multiplicatif pour obtenir les puissances
                -- successives de x
puissance: Réel  -- abs(n). On a : facteur^puissance = x^n
```

On peut maintenant formaliser notre raisonnement sous la forme du raffinement suivant.

```

1  R0 : Afficher la puissance entière d'un réel
2
3  R1 : Raffinage De « R0 »
4      | Saisir avec contrôle les valeurs de x et n      x: out Réel; n: out Entier
5      | { (x <> 0) Ou (n > 0) }
6      | Calculer x à la puissance n                    x, n: in ; xn: out Réel
7      | Afficher le résultat                            xn: in Réel
8
9  R2 : Raffinage De « Saisir avec contrôle les valeurs de x et n »
10     | Répéter
11     | | Saisir la valeur de x et n                    x: out Réel; n: out Entier
12     | | Contrôler x et n                              x, n: in ; valide: out Booléen
13     | Jusqu'À valide                                  valide: in
14
15  R1 : Raffinage De « Calculer x à la puissance n »
16     | Si x = 0 Alors
17     | | xn := 0
18     | Sinon
19     | | Déterminer le facteur multiplicatif et la puissance
20     | | | x, n: in ;
21     | | | facteur out Réel ;
22     | | | puissance: out Entier
23     | | Calculer xn par itération (accumulation)
24     | | | n, facteur, puissance: in ; xn : out
25     | FinSi
26
27  R3 : Raffinage De « Calculer xn par itération (accumulation) »
28     | xn <- 1;
29     | Pour i <- 1 Jusqu'À i = puissance Faire
30     | | { Invariant : xn = facteuri }
31     | | xn <- xn * facteur
32     | FinPour

```

On peut alors en déduire l'algorithme suivant :

```

1  Algorithme puissance
2
3      -- Afficher la puissance entière d'un réel
4
5  Variables
6      x: Réel          -- valeur réelle lue au clavier
7      n: Entier        -- valeur entière lue au clavier
8      valide: Booléen  --  $x^n$  peut-elle être calculée
9      xn: Réel         -- x à la puissance n (en fait  $x^{(i-1)}$ )
10     facteur: Réel     -- facteur multiplicatif pour obtenir
11                       -- les puissances successives
12     exposant: Entier  -- abs(n). On a  $\text{facteur}^{\text{exposant}} = x^n$ 
13     i: Entier         -- variable de boucle
14
15  Début
16     -- Saisir avec contrôle les valeurs de x et n
17     Répéter
18     -- saisir la valeur de x et n
19     Écrire("x = ")
20     Lire(x)
21     Écrire("n = ")
22     Lire(n)
23

```

```

24     -- contrôler x et n
25     valide <- VRAI
26     Si x = 0 Alors
27         Si n = 0 Alors
28             ÉcrireLn("x et n sont nuls.  $x^n$  est indéterminée.")
29             valide <- FAUX
30         SinonSi n < 0 Alors
31             ÉcrireLn("x nul et n négatif.  $x^n$  n'a pas de sens.")
32             valide <- FAUX
33         FinSi
34     FinSi
35
36     Si Non valide Alors
37         ÉcrireLn(" Recommencez !")
38     FinSi
39     JusquÀ valide
40     { x <> 0 Ou n >= 0 }
41
42     -- Calculer x à la puissance n
43     -- Déterminer le facteur multiplicatif et l'exposant
44     Si n >= 0 Alors
45         facteur <- x
46         exposant <- n
47     Sinon
48         facteur <- 1/x
49         exposant <- -n
50     FinSi
51
52     -- Calculer xn par itération (accumulation)
53     xn <- 1;
54     Pour i <- 1 JusquÀ i = exposant Faire
55         { Invariant :  $xn = \text{facteur}^i$  }
56         xn <- xn * facteur
57     FinPour
58
59     -- Afficher le résultat
60     ÉcrireLn(x, "^", n, " = ", xn)
61 Fin.
```

Exercice 4 : Amélioration du calcul de la puissance entière

Améliorer l'algorithme de calcul de la puissance (`puissance_mieux.adb`) en remarquant que :

$$x^n = \begin{cases} (x^2)^p & \text{si } n = 2p \\ (x^2)^p \times x & \text{si } n = 2p + 1 \end{cases}$$

Ainsi, pour calculer 3^5 , on peut faire $3 * 9 * 9$ avec bien sûr $9 = 3^2$.

Solution : Nous nous appuyons sur les mêmes variables que pour le calcul « naturel » de la puissance (exercice 3). Nous allons continuer à calculer x^n par accumulation. Nous avons l'invariant suivant :

$$x^n = xn * \text{facteur}^{\text{puissance}}$$

Lors de l'initialisation, cet invariant est vrai (xn vaut 1 et facteur et puissance sont tels qu'ils valent x^n).

D'après la formule donnée dans l'énoncé, à chaque itération de la boucle, deux cas sont à envisager :

— soit puissance est paire. On peut alors l'écrire $2 * p$. On a alors :

$$\begin{aligned} x^n &= xn * \text{facteur}^{\text{puissance}} \\ &= xn * \text{facteur}^{(2 * p)} \\ &= xn * (\text{facteur}^2)^p \end{aligned}$$

On peut donc faire :

```
facteur <- facteur * facteur
puissance <- puissance Div 2    -- car p = puissance Div 2
```

On constate que l'invariant est préservé d'après les égalités ci-dessus et la puissance a strictement diminué (car divisée par 2).

— soit puissance est impaire. On peut alors l'écrire $2 * p + 1$. On a alors :

$$\begin{aligned} x^n &= xn * \text{facteur}^{\text{puissance}} \\ &= xn * \text{facteur}^{(2 * p + 1)} \\ &= xn * (\text{facteur} * \text{facteur}^{(2 * p)}) \\ &= (xn * \text{facteur}) * \text{facteur}^{(2 * p)} \end{aligned}$$

On peut donc faire :

```
xn <- xn * facteur
puissance <- puissance - 1
```

On constate que l'invariant est préservé d'après les égalités ci-dessus et la puissance a strictement diminué (car diminuée de 1).

On peut alors en déduire le raffinement suivant :

```
1  R3 : Raffinage De « Calculer xn par itération (accumulation) »
2      | xn <- 1;
3      | TantQue puissance > 0 Faire
4      | | { Variant : puissance }
5      | | { Invariant :  $x^n = xn * \text{facteur}^{\text{puissance}}$  }
6      | | Si puissance Div 2 = 0 Alors { puissance = 2 * p }
7      | | | puissance <- puissance Div 2
8      | | | facteur <- facteur * facteur
9      | | Sinon { puissance = 2 * p + 1 }
10     | | | puissance <- puissance - 1
11     | | | xn <- xn * facteur
12     | | FinSi
13     | FinTQ
```